UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

Sistemas Distribuidos

Clase 09 — Fronteras de arquitectura: monolito modular, servicios, eventos, outbox y sagas

Departamento de Computación
Ingeniería Informática – Comisión A – Pilar
Equipo docente – 2026

La historia que guiará toda la clase

NÚCLEO

Asteria Online es un mundo virtual medieval multijugador. Hoy no diseñaremos el juego completo: seguiremos una única compra para descubrir cuándo una separación arquitectónica ayuda y cuándo solo agrega red, fallas y operación.



Objeto concreto

Sofía intenta comprar la pieza única `EspadaSolar#17`. La compra involucra mercado, inventario, pago y efectos posteriores; el objeto no puede terminar con dos propietarios.

El alcance: qué entra y qué queda fuera

Analizaremos

- catálogo y publicación de la pieza;
- reserva y propiedad en inventario;
- autorización del pago;
- ranking y notificaciones posteriores;
- equipos, puesta en producción, datos y fallas.

No diseñaremos hoy

- física y renderizado del mundo;
- protocolo de movimiento en tiempo real;
- emparejamiento de jugadores (matchmaking);
- detección de trampas (anti-cheat);
- infraestructura gráfica del cliente.

Acotar el caso evita usar palabras grandes sobre un sistema indefinido. Cada decisión se justificará sobre esta compra y sus fallas.

NÚCLEO

La compra de `EspadaSolar#17`: historia completa

CASO CONDUCTOR



No puede romperse

- la pieza tiene como máximo un propietario;
- no se cobra dos veces el mismo intento;
- una compra confirmada no desaparece.

Puede demorarse

- el ranking;
- la notificación;
- una vista de analítica;
- el historial derivado.

Vocabulario intuitivo I: dominio, capacidad e invariante

VOCABULARIO PREVIO

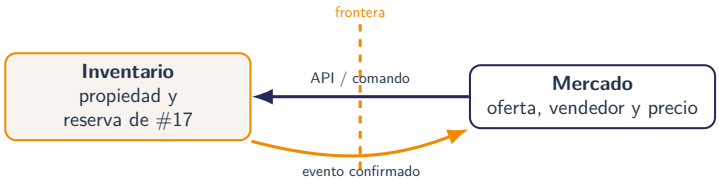
Concepto	Idea intuitiva	Ejemplo en Asteria Online
Dominio	Parte de la realidad con problemas, reglas y palabras propias.	El comercio de objetos incluye ofertas, vendedores, compradores, precios y ventas.
Capacidad	Responsabilidad estable que el negocio necesita sostener. Agrupa reglas, datos y varias operaciones; no es una acción aislada.	**Gestionar la propiedad**: reservar, transferir, liberar y consultar quién posee una pieza.
Invariante	Regla que debe seguir siendo verdadera en todo estado válido.	`EspadaSolar#17` tiene como máximo un propietario y un intento no se cobra dos veces.

Primero identificamos una responsabilidad y la regla que debe preservar. Recién después decidimos si se implementará como módulo, proceso o servicio.

Vocabulario intuitivo II: frontera y dato autoritativo

Frontera

Límite explícito que separa dos ámbitos del sistema. Indica qué responsabilidades, reglas y datos quedan de cada lado y por qué puertas se permite interactuar.



Inventario conserva el dato autoritativo sobre el propietario. Mercado no lo modifica directamente: cruza la frontera por una operación permitida o consume un hecho ya confirmado.

Vocabulario intuitivo III: autoridad y responsabilidad

VOCABULARIO PREVIO

Concepto	Definición de trabajo	En Asteria
Autoridad	Componente con la última palabra sobre un hecho de negocio determinado.	Inventario decide quién es el propietario válido de `EspadaSolar#17`.
Responsabilidad (ownership)	Equipo que puede cambiar reglas y datos y que responde por métricas, incidentes y recuperación.	El equipo de Inventario mantiene el invariante de propiedad y atiende una duplicación.
Dato autoritativo	Registro reconocido como verdad oficial para decidir.	`propietario(#17)=Sofía`; Ranking mantiene una vista derivada.

Autoridad responde **quién decide el hecho**; responsabilidad responde **quién sostiene esa decisión durante toda la vida del sistema**.

Vocabulario intuitivo IV: versión y puesta en producción

VOCABULARIO PREVIO

Término	Definición de trabajo	Pregunta observable
Unidad de publicación	Conjunto de componentes que se publica, actualiza o retira como una misma versión.	¿Mercado e Inventario deben salir siempre juntos?
Puesta en producción	Hacer que una versión atienda tráfico real.	¿Ranking puede publicar una versión sin volver a publicar Inventario?
Vuelta atrás de versión (rollback)	Retirar una versión problemática y regresar tráfico o software a una versión conocida.	¿Podemos volver sin interrumpir las compras?

Volver el software a una versión anterior no deshace pagos, eventos ni transferencias de propiedad ya confirmadas. Esos efectos requieren reparación de negocio.

Vocabulario intuitivo V: autonomía

Definición de trabajo

La **autonomía** es el grado en que una parte puede cambiar, publicar una versión, escalar o recuperarse sin obligar al resto a hacerlo al mismo tiempo.

Dimensión	Pregunta observable en Asteria
Cambio	¿Inventario puede modificar su lógica sin exigir cambios simultáneos en Mercado?
Puesta en producción	¿Ranking puede salir o volver atrás sin volver a publicar Inventario?
Escalado	¿Ranking puede agregar recursos durante un torneo sin escalar también Pagos?
Recuperación	¿Una falla de Ranking puede resolverse sin detener la compra?

Separar no crea autonomía por sí solo: una base compartida, versiones coordinadas o un equipo sin capacidad operativa pueden mantener el acoplamiento.

Dos formas de ejecutar una secuencia de trabajo

Cadena sincrónica

La petición espera una respuesta antes de continuar:

$$A \rightarrow B \rightarrow C \rightarrow D$$

Si C tarda o falla, A todavía no puede terminar.

Continuación asíncrona

Un hecho durable permite que otros continúen después:

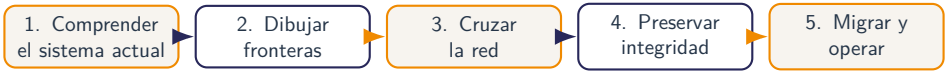
$$A \rightarrow log \Rightarrow \{C, D, E\}$$

A no espera a todos los lectores.

No son estilos universales. Una autorización de pago puede necesitar respuesta inmediata; actualizar ranking normalmente no.

Cómo vamos a recorrer la historia

NÚCLEO



Pregunta que une la clase

¿Qué beneficio concreto obtiene cada separación y qué nueva falla, latencia, dato duplicado o responsabilidad operacional introduce?

Evidencias de aprendizaje que produciremos

NÚCLEO

Durante la clase

- delimitar una capacidad y su dato autoritativo;
- transformar una llamada local en una cadena sincrónica;
- mover un crash por un dual write;
- localizar qué invariante deja de ser local.

Al cerrar

- justificar una frontera con evidencia;
- diseñar outbox e idempotencia;
- modelar una saga durable;
- proponer una migración reversible y métricas.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

De una regla local a una coordinación entre fronteras

Sistemas Distribuidos

Puente desde la Clase 08: ¿cuándo hace falta un protocolo?

NÚCLEO

Regla local

Si una sola autoridad posee el dato, puede preservar el invariante dentro de una transacción local: Inventario decide quién posee `EspadaSolar#17`.

Regla distribuida

Si la decisión atraviesa procesos independientes, hace falta un **protocolo de coordinación**: una secuencia acordada de mensajes, estados durables, IDs, timeouts, retries y reglas de recuperación.

Mover una frontera no elimina la regla de negocio: transforma una transacción local en un protocolo que debe sobrevivir fallas parciales.

Diagnóstico de entrada: vote antes de discutir

ACTIVIDAD

1. Un monolito termina necesariamente como una maraña de dependencias.
2. Un microservicio es una clase de negocio expuesta por HTTP.
3. Separar procesos vuelve independientes a los equipos automáticamente.
4. Publicar un evento después de guardar en la base resuelve el dual write.
5. Una compensación de saga equivale a deshacer perfectamente la operación.

Consigna

Vote V/F. No busque todavía la definición perfecta: anote qué palabra le genera desconfianza.

Módulo, proceso, servicio y microservicio no son sinónimos

VOCABULARIO JUSTO A TIEMPO

Término	Qué es	Ejemplo
Módulo	frontera de código en una aplicación; expone una interfaz interna y oculta detalles.	módulo Inventario dentro del monolito.
Proceso	programa en ejecución, con memoria y fallas propias.	una instancia ejecutando el servicio de Inventario.
Servicio	capacidad accesible mediante un contrato, normalmente a través de red.	API —interfaz explícita de operaciones permitidas— para reservar o consultar una pieza.
Microservicio	servicio de una capacidad coherente, con despliegue y operación razonablemente independientes.	Ranking separado, operado por un equipo y alimentado por eventos.

Punto de partida: Asteria funciona como una aplicación modular

CASO CONDUCTOR

una aplicación que se publica como una unidad



Lo que funciona bien

- llamadas y transacciones locales;
- diagnóstico y vuelta atrás unificados;
- cambios cruzados simples;
- fronteras fáciles de refactorizar.

Señales de que empieza a quedar corto

- Ranking provoca picos de carga;
- Pagos exige controles propios;
- Combate crece por zonas;
- los equipos se bloquean para publicar cambios.

La escala de Asteria cambia la escena

CASO CONDUCTOR

Supuesto para seguir la historia

Sofía ya no compra en un mundo con pocos jugadores. Hay muchos compradores en distintas regiones, torneos que disparan consultas a Ranking y más ofertas que se publican a la vez.

Cambian las cargas

Catálogo y Ranking reciben muchas lecturas; las compras modifican ofertas y reservas. Esas capacidades no crecen al mismo ritmo.

La regla sigue siendo la misma

Sofía y otro jugador pueden intentar comprar #17 a la vez. Inventario debe decidir una única reserva válida antes de transferirla.

Conclusión: más usuarios exigen escalar; separar capacidades requiere un límite medido.

Primeras respuestas sin separar capacidades

Presión observable	Respuesta dentro de la aplicación modular	Costo y límite
Más solicitudes	réplicas del proceso y balanceo de tráfico	se replica toda la aplicación; la base puede seguir saturada
Muchas lecturas	caché de catálogo y vistas de Ranking	invalidación y lecturas atrasadas; la compra revalida la oferta
Torneos exigentes	diferir cálculos de Ranking y limitar su uso de recursos	cola, atraso y trabajo de recuperación; puede persistir la contención
Compras simultáneas	reserva condicional en la base de Inventario	contención en la pieza; medir espera y conflictos

Criterio: separar si estas medidas no alcanzan para cumplir el p99 de compra, aislar la carga o mejorar el tiempo hasta producción.

DEMOSTRATOR

Antes de separar: ¿qué problema concreto queremos resolver?

CRITERIO DE DECISIÓN

Necesidad observable	Límite del diseño actual	Evidencia
Carga diferente	el torneo de Ranking vuelve lenta la compra	CPU, cola y p99 por capacidad
Ritmos de cambio	un cambio de Ranking espera la publicación global	tiempo hasta producción y coordinaciones
Aislamiento de fallas	una falla secundaria afecta el camino de compra	alcance del incidente
Seguridad o regulación	Pagos necesita permisos, auditoría y secretos específicos	incidentes y controles
Responsabilidad de equipos	nadie opera una parte sin pedir permiso a otros	bloqueos y tickets cruzados

Una **necesidad concreta de separación** es una limitación observable que la modularidad interna ya no resuelve suficientemente. Sin problema, evidencia e hipótesis de mejora, distribuir es solo agregar complejidad.

La analogía del edificio: mapeo explícito

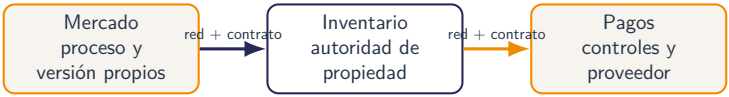
edificio = una versión publicada como conjunto



Edificio	Habitación	Puerta	Pared
Aplicación que se publica como una unidad.	Módulo con responsabilidad definida.	API interna permitida.	Regla que impide tocar internals o tablas ajenas.

Compartir un proceso simplifica comunicación y transacciones. El problema no es publicar los módulos juntos, sino perder las fronteras internas que separan responsabilidades y regulan cómo interactúan.

¿Qué significa separar en este caso?



Puede habilitar

- publicar versiones por separado;
- escalar solo la parte exigida;
- limitar el alcance de una falla;
- asignar responsabilidad completa.

No lo garantiza

- una base compartida mantiene acoplamiento;
- versiones coordinadas eliminan independencia;
- contratos frágiles obligan a cambiar juntos;
- sin responsable de incidentes no hay autonomía operativa.

Separar es una hipótesis: aceptamos red y fallas parciales para intentar obtener una independencia medible. Debe comprobarse con evidencia.

Predicción individual: ¿dónde pondría la primera frontera?

ACTIVIDAD

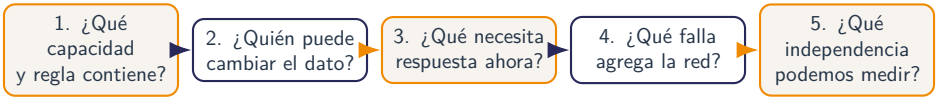
Elija una capacidad de Asteria que mantendría dentro de la aplicación y una que separaría. Para cada una escriba:

1. una necesidad concreta;
2. una autonomía que espera obtener;
3. una falla nueva que aparecería.

Guarde la respuesta: volveremos después de estudiar responsabilidad, comunicación, outbox y sagas.

Cinco preguntas antes de dibujar servicios

NÚCLEO



UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

Primero el significado: contextos y responsabilidades

Contexto delimitado: una misma palabra puede nombrar cosas distintas

NÚCLEO

Palabra	Contexto	Significado exacto en ese contexto
Objeto	Catálogo	tipo de pieza: nombre, rareza y atributos base de `EspadaSolar`
Objeto	Inventario	instancia única #17: propietario, estado y versión
Objeto	Mercado	oferta de venta: vendedor, precio, moneda y estado de publicación
Objeto	Combate	arma equipada: estadísticas efectivas, durabilidad y tiempo de reutilización

Un **bounded context** o **contexto delimitado** es una frontera dentro de la cual el vocabulario, el modelo y las reglas mantienen un significado coherente.

Por qué no conviene un único objeto universal

NÚCLEO



- Catálogo puede cambiar atributos base sin modificar el historial de propiedad.
- Mercado puede cerrar una oferta sin borrar la instancia del inventario.
- Combate necesita una vista rápida, pero no debe convertirse en dueño de la pieza.

Traducción explícita

Los contextos intercambian IDs, comandos y eventos; no comparten silenciosamente un objeto universal mutable.

Un contexto delimitado no obliga a crear un proceso

Dentro de un monolito

- módulo Inventario;
- API interna;
- esquema o espacio de nombres protegido;
- pruebas de arquitectura;
- transacción local.

Como servicio separado

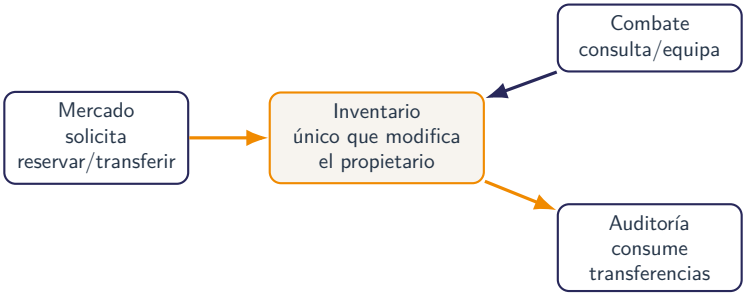
- proceso propio;
- API o mensajes de red;
- publicación de versión y escalado propios;
- datos bajo autoridad exclusiva;
- nuevas fallas parciales.

Primero se descubre la frontera semántica. La separación física se decide después, cuando existe una necesidad concreta que justifica asumir el costo adicional de distribuir.

NÚCLEO

Responsabilidad: quién puede decidir la verdad sobre #17

NÚCLEO



Regla

Inventario define cuándo la pieza puede cambiar de dueño.

Estado

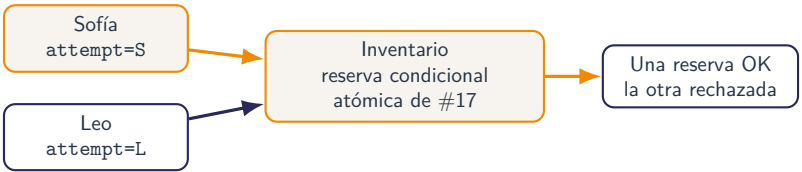
Solo Inventario modifica el dato autoritativo.

Equipo

Cambia la regla, observa métricas y atiende incidentes.

Sofía y Leo compiten por la misma espada

CASO CONDUCTOR



La decisión que protege el invariante

Inventario cambia `DISPONIBLE` a `RESERVADA(attempt)` solo si `#17` todavía está disponible. La transacción serializa la competencia: una petición obtiene éxito y la otra debe elegir otra pieza.

Timeout: Sofía conserva el ID de intento al reintentar, para evitar otra compra lógica.

Cuando el invariante cruza fronteras aparece un protocolo

NÚCLEO



Qué significa protocolo aquí

Secuencia explícita de mensajes, estados durables y reglas para continuar, reintentar o reparar cuando los participantes no comparten una transacción y alguno puede fallar.

- identidad de cada intento;
- autoridad de cada dato;
- estado que se guarda antes de responder;
- significado de un timeout;
- política de reintento;
- reparación de efectos parciales.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Aplicación modular o servicios separados: simplicidad o independencia medible

Sistemas Distribuidos

Monolito modular: una solución, no una etapa vergonzosa

Dimensión	Monolito modular	Aplicación enmarañada
Dependencias	explícitas y verificables	cualquier parte llama a cualquier otra
Datos	acceso mediante APIs internas	tablas compartidas sin autoridad
Cambios	contratos y pruebas internas	efectos laterales impredecibles
Responsabilidad	responsables claros	todos pueden cambiar todo
Publicación	conjunta por decisión	conjunta porque nada puede separarse

Compartir un proceso simplifica comunicación y transacciones. El problema aparece cuando el código se vuelve una maraña sin límites internos, no por el mero hecho de publicar los módulos juntos.

NÚCLEO

Cuándo conviene mantener Asteria modular y unida

NÚCLEO

- El dominio todavía cambia y las fronteras no están maduras.
- El equipo es pequeño y modifica varios módulos en conjunto.
- La compra necesita invariantes que caben en una transacción local.
- La carga de los componentes todavía es parecida.
- El equipo aún no posee herramientas y guardias para operar muchos servicios.

Ventaja de esperar

Refactorizar una frontera dentro del proceso es más barato que cambiarla cuando ya es una API de red, un esquema de eventos almacenado y una responsabilidad operativa independiente.

Qué independencia puede justificar una separación

Independencia buscada	Evidencia concreta
Publicar versiones	Ranking sale o vuelve atrás sin coordinar Mercado ni Inventario
Escalar	Ranking absorbe un torneo sin multiplicar recursos de Pagos
Aislar fallas	una caída de Chat no interrumpe la compra
Evolucionar datos	Inventario cambia internals sin acceso directo de otros componentes
Responsabilidad de equipo	un grupo decide cambios y atiende incidentes de la capacidad

Separar no crea independencia automáticamente. Una base compartida, versiones coordinadas o contratos frágiles pueden dejar el sistema igual de acoplado, pero con red en el medio.

Primera frontera: separar Ranking

CASO CONDUCTOR

Síntoma observado

El torneo carga CPU y base de datos compartidas: el p99 de compra sube. Ranking ya se calcula después de comprar, pero comparte recursos y versión publicada con Mercado.

Decisión y beneficio esperado

Ranking procesa compras confirmadas. Puede escalar y publicar versiones sin cargar el flujo de compra.

Costo que aceptamos

Entrega durable del evento, atraso de la vista, duplicados, reconstrucción, observabilidad y guardia del nuevo servicio.

Contrato: Mercado publica `PurchaseConfirmed` con outbox e idempotencia.

Sofía compra desde otra región

CASO CONDUCTOR

Lecturas cercanas al jugador

Catálogo y Ranking pueden usar cachés o réplicas regionales. Sofía ve la oferta con menos latencia, aunque la copia puede estar atrasada.

Escritura bajo una autoridad

Antes de confirmar #17, Mercado pide a Inventario reservarla. Esa operación necesita una decisión coordinada y puede sumar tiempo de red entre regiones.

Una oferta visible en caché no garantiza disponibilidad. La reserva autoritativa decide si Sofía gana; acercar lecturas no habilita dos propietarios.

De llamada local a cadena sincrónica



Definición

Una **cadena sincrónica** es un flujo donde la petición original no puede terminar hasta recibir las respuestas requeridas de varias dependencias sucesivas.

- las latencias se acumulan;
- un timeout remoto es ambiguo;
- la disponibilidad del flujo depende de todos los participantes requeridos.

El costo adicional de distribuir: qué aparece al cruzar la red

NÚCLEO

Comunicación latencia y timeout

Entrega retry e idempotencia

Datos consistencia y copias

Contratos compatibilidad

Operación trazas y guardias

Seguridad identidad y secretos

Regla económica

Aceptamos este costo solo cuando resuelve una necesidad concreta y la mejora puede medirse: menor tiempo hasta producción, escala separada, menor alcance de incidentes o reglas mejor aisladas.

La red no elimina el acoplamiento: puede transformarlo en timeouts, contratos, duplicados y coordinación entre equipos.

Disponibilidad compuesta: una aproximación útil, no una ley

NÚCLEO

Si cuatro dependencias independientes son obligatorias y cada una está disponible el 99.9%:

$$A_{flujo} \approx 0.999^4 \approx 0.996$$

Qué muestra	Qué omite
Cada dependencia requerida puede bloquear el camino crítico.	Fallas correlacionadas, red, colas, retries, cachés, degradación y dependencia condicional.

Más procesos no implican más disponibilidad. El objetivo es retirar dependencias no críticas o permitir degradación, no multiplicar cajas.

La ruta crítica de la compra de Sofía

CASO CONDUCTOR

Paso antes de responder éxito	Resultado requerido	Costo si cruza procesos
Mercado valida la oferta	intento aceptado	llamada y estado durable
Inventario reserva #17	exclusión de otros compradores	red, timeout y posible contención
Pagos autoriza	respuesta del proveedor	latencia e incertidumbre de pago
Inventario transfiere	Sofía queda propietaria	otra llamada y recuperación si falla
Mercado confirma	compra y aviso pendientes en un COMMIT local	escritura local y posible reintento

Costo acumulado: se suman las esperas obligatorias y los reintentos. Ranking y Notificaciones quedan fuera del camino crítico, con atraso medible.

Muchos servicios, pero una sola unidad de cambio

ANTIPATRÓN



misma base + versiones coordinadas + modelos compartidos + cadena RPC obligatoria

Por qué se usa la expresión “monolito distribuido”

Es una etiqueta crítica y paradójica: físicamente hay varios procesos y red, pero los cambios, datos y operación siguen comportándose como una única pieza inseparable.

Se pagan latencia, timeouts y observabilidad distribuida sin obtener independencia. El nombre describe un antipatrón; no propone una arquitectura deseable.

Casos reales: la forma depende de la necesidad

Shopify: reforzar límites internos

Una aplicación grande puede mejorar módulos y dependencias dentro del mismo proceso mientras publicar todo junto siga siendo aceptable.

Netflix: servicios con plataforma madura

La separación puede tener sentido cuando cargas, equipos y ciclos son distintos y existen herramientas para publicar, observar y recuperar cada servicio.

Copiar la forma sin compartir la necesidad y la capacidad operativa produce complejidad sin beneficio. Primero se declara el problema; después se elige la estructura.

Fuente / referencia: Shopify Engineering; Netflix TechBlog; Fowler, Monolith First y Microservice Trade-Offs.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Cruzar la frontera: RPC, comandos, consultas y eventos

Sistemas Distribuidos

Llamada local y RPC se parecen en el código, no en la falla

RPC

Remote Procedure Call o **llamada a procedimiento remoto**: contrato que hace parecer una interacción de red como una llamada a una función. Simplifica la interfaz; no elimina la incertidumbre distribuida.

Llamada local

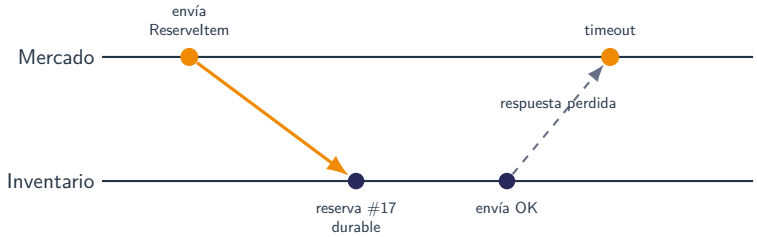
- mismo proceso;
- error inmediato y visible;
- una pila y un entorno de ejecución;
- transacción local posible.

RPC

- serialización y red;
- petición o respuesta pueden perderse;
- el receptor puede ejecutar y el emisor observar timeout;
- requiere identidad, retry y observabilidad.

Timeout ambiguo: ¿Inventario reservó la espada?

EJECUCIÓN ADVERSARIAL



Lo único que Mercado sabe

No recibió respuesta dentro de su plazo. No sabe si Inventario no recibió, falló antes de reservar, reservó y perdió la respuesta, o sigue lento.

El retry necesita una clave idempotente estable y un resultado durable para no crear una segunda reserva lógica.

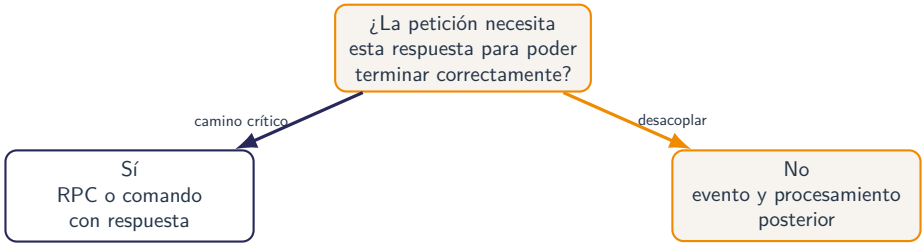
Comando, evento y consulta: tres contratos distintos

NÚCLEO

Mensaje	Semántica	Ejemplo Asteria
Comando	pedir al responsable autoritativo que intente una acción; puede rechazar	`ReserveItem(item_17)`
Evento	declarar un hecho que ya ocurrió	`ItemReserved(item_17)`
Consulta	pedir información sin cambiar el estado autoritativo	`GetItemOwner(item_17)`

Un nombre en pasado no convierte un comando en evento. La diferencia está en quién decide y si el receptor puede rechazar la acción.

Elegir RPC o evento desde la necesidad del flujo



Ejemplos sincrónicos

reservar la pieza, autorizar pago, consultar una regla necesaria.

Ejemplos asíncronos

notificación, ranking, analítica y otras vistas derivadas.

Necesidad: varios componentes deben enterarse sin bloquear la compra

PROBLEMA



Problema

Mercado no debería esperar a todos los lectores ni conocer cada uso futuro.

Necesidad

Conservar un aviso que otros puedan leer, reintentar y procesar a su propio ritmo.

Intuición: un evento es un aviso durable sobre un hecho

Ejemplo

`PurchaseConfirmed(purchase_id=42, item=17, buyer=Sofía)` informa que Mercado ya confirmó la compra tras conocer la transferencia. Describe pasado confirmado; no ordena una acción futura.

Qué permite

- lectores independientes;
- replay después de una caída;
- nuevas vistas sin modificar Inventario.

Qué exige

- identidad estable;
- significado inequívoco;
- formato compatible;
- política de duplicados y orden.

Si el mensaje pretende que un destinatario haga algo, es un comando, aunque viaje por un broker.

Contrato de evento: la interfaz pública que queda almacenada

FORMALIZACIÓN

Definición

El **contrato de un evento** especifica qué hecho representa, qué campos transporta, cómo se identifica, qué orden puede suponerse y cómo evoluciona sin romper consumidores.

Parte	**En** `PurchaseConfirmed`
Identidad	`event_id`, `purchase_id` y versión del evento
Semántica	Mercado confirmó la compra tras la transferencia; no es una intención pendiente
Esquema	nombres, tipos y obligatoriedad de campos
Evolución	reglas para agregar o retirar campos sin romper lectores viejos
Entrega	posibilidad de duplicado, orden relevante por compra y replay

Se parece a una API porque otros componentes dependen de esa interfaz. La diferencia es que queda almacenada: consumidores nuevos o atrasados leen eventos producidos por versiones anteriores.

Ventajas y costos de publicar un contrato de evento

TRADE-OFF

Ventajas

- desacopla productor y lectores;
- permite replay y reconstrucción;
- agrega lectores sin cambiar Inventario;
- retira efectos secundarios del camino crítico.

Costos

- duplicados y atraso;
- compatibilidad durante años;
- observabilidad de lag y fallas;
- mala semántica se propaga a todos.

Identidad: cuándo dos registros representan el mismo hecho. Semántica: qué afirma. Evolución: cómo cambia el formato sin romper la historia almacenada.

UNIVERSIDAD AUSTRAL
INGENIERÍA
Departamento de Computación

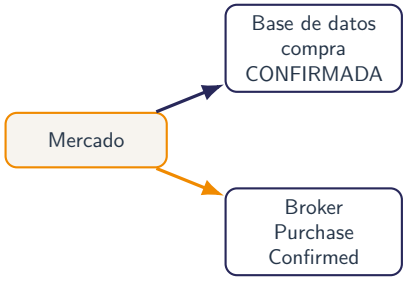
Preservar integridad: dos escrituras, outbox e idempotencia

Dual write: una intención, dos sistemas sin transacción común

La necesidad

Mercado debe:

1. confirmar la compra en su base de datos (DB);
2. publicar `PurchaseConfirmed` para Ranking y Notificaciones.



No existe un único COMMIT que abarque automáticamente la base local y el broker remoto.

Dos órdenes, dos ventanas de falla

Base de datos primero

1. confirmar compra en la base;
2. **crash**;
3. evento nunca publicado.

Resultado: compra autoritativa sin notificación ni ranking.

Broker primero

1. publicar evento;
2. **crash o reversión de la transacción local**;
3. la base no confirma la compra.

Resultado: consumidores reaccionan a una compra inexistente.

Cambiar el orden no elimina la ventana: solo cambia qué mitad queda sola.

Intuición de outbox: dejar el aviso en la misma operación local

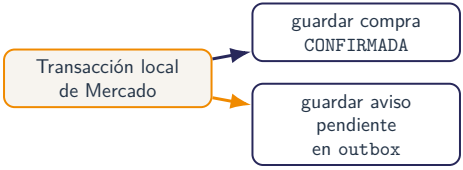


Imagen mental

Después de conocer la transferencia de Inventario, Mercado anota la compra confirmada y el aviso que deberá enviar. Si la transacción local confirma, existen ambos; si aborta, no existe ninguno.

La outbox evita olvidar la intención de publicar si el proceso cae después del commit local. Todavía hace falta un proceso que publique el aviso.

INTUICIÓN

Definición formal: transactional outbox

Definición

Una **transactional outbox** es una tabla o colección persistente donde el servicio inserta un evento pendiente dentro de la misma transacción local que modifica el estado de negocio.



- el relay puede leer por polling o CDC;
- la publicación suele ser al menos una vez;
- el consumidor deduplica mediante `event_id` o inbox transaccional.

Qué resuelve outbox y qué deja abierto

TRADE-OFF

Resuelve

- estado sin intención durable de publicar;
- crash entre commit local y registro del aviso;
- recuperación del trabajo pendiente.

No resuelve sola

- duplicados del relay;
- COMMIT global con Inventario o Pagos;
- consumidores no idempotentes;
- atraso y crecimiento de la tabla.

Métricas mínimas: filas pendientes, edad de la más antigua, reintentos y tiempo extremo a extremo hasta el consumidor.

Sofía ve éxito; Ranking todavía puede estar atrasado

CASO CONDUCTOR

1. Inventario confirma la transferencia. Mercado guarda `CONFIRMADA` y `PurchaseConfirmed` pendiente en un mismo COMMIT local.
2. Sofía recibe la confirmación basada en ese estado autoritativo. El relay aún puede estar detenido y Ranking mostrar la puntuación anterior.
3. El relay publica o reintenta; Ranking deduplica por `event_id` y actualiza su proyección.

Beneficio

Una falla de Ranking no borra la compra confirmada y la proyección puede recuperarse.

Costo observable

Medir edad de la outbox, atraso de Ranking, duplicados y tiempo de reconstrucción.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Coordinar la compra: saga y reparación

Sistemas Distribuidos

Tres autoridades independientes participan en la compra

Autoridad	Hecho que decide	Por qué es independiente
Mercado	intento y estado de la oferta	posee su estado, transacción y fallas
Inventario	reserva y propietario de #17	confirma en su propia base y reinicia aparte
Pagos	autorización o rechazo del pago	depende de un proveedor y no comparte COMMIT con Inventario

Independencia en esta historia

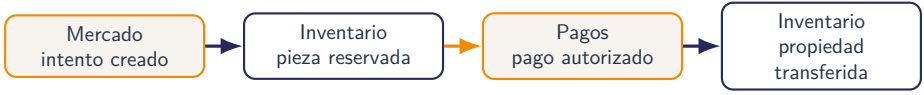
Cada autoridad puede confirmar, fallar, reintentar o recuperarse sin que las otras participen de la misma transacción atómica. Ninguna puede deshacer por sí sola todo el proceso.

Por qué aparece una saga en Asteria

NECESIDAD

Problema

La compra atraviesa Mercado, Inventario y Pagos. Cada autoridad confirma localmente, pero no existe un COMMIT único que abarque a las tres.



Necesitamos conservar el estado del proceso y decidir qué hacer si un paso tarda, se duplica, queda incierto o falla después de que otros ya confirmaron.

Intuición: una saga es un itinerario durable de negocio

Imagen mental

La compra es un viaje con varias paradas. En cada parada se anota durablemente dónde quedó, qué respuesta recibió y cuál es el próximo paso permitido.



Si el proceso reinicia, no vuelve a empezar a ciegas: lee el estado durable y continúa, consulta o repara desde allí.

Definición formal: saga

Definición

Una **saga** coordina una operación mediante transacciones locales. Conserva estados durables y define reintentos, consultas, timeouts y compensaciones para resultados parciales.

Registra

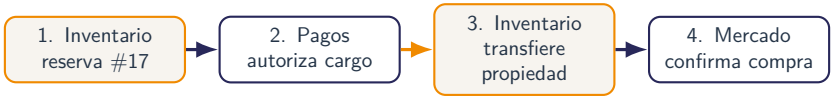
- `saga_id` e intento;
- estado y deadline;
- comandos y respuestas;
- último resultado durable.

Decide

- continuar;
- consultar o reintentar;
- compensar;
- pasar a revisión manual.

Camino feliz: qué confirma cada participante

NÚCLEO

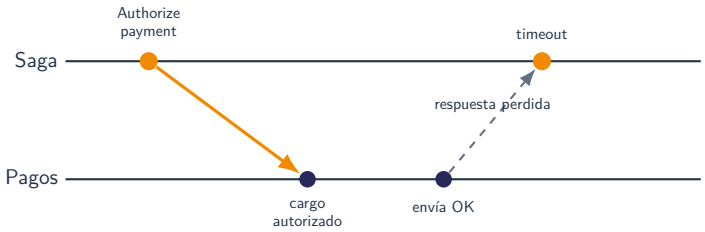


Pregunta previa

¿En qué estado puede responderse “compra confirmada” y qué evidencia durable demuestra que la pieza y el cargo quedaron coherentes?

Falla realista: el pago puede quedar incierto

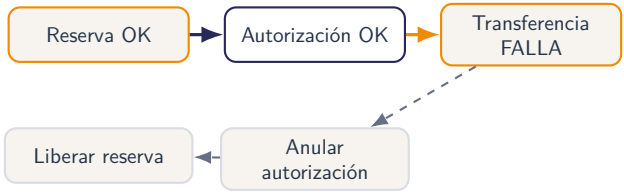
EJECUCIÓN ADVERSARIAL



Reintentar ciegamente puede cobrar dos veces. La saga necesita un payment_attempt_id, consulta de estado y un estado durable `PAYMENT_UNKNOWN`.

Falla posterior: pago autorizado, transferencia imposible

NÚCLEO



Estado global

La saga pasa a `COMPENSATING`; solo termina cuando conoce el resultado durable de cada compensación o deriva el caso a intervención manual. Si el cargo ya se capturó, puede hacer falta un reembolso.

Tres formas de “volver atrás” que no son equivalentes

Mecanismo	Qué hace	Ejemplo
Rollback transaccional	aborta cambios locales aún no confirmados	la base revierte una transacción sin COMMIT
Vuelta atrás de versión	devuelve tráfico o software a una versión anterior	se retira una versión defectuosa de Ranking
Compensación de negocio	ejecuta una nueva acción para reparar un efecto confirmado	anular autorización y liberar la reserva

Volver una versión de software no borra pagos, eventos ni cambios de propiedad ya producidos. Esos efectos requieren acciones explícitas.

Compensación: reparar el negocio sin borrar la historia

NÚCLEO

Definición

Una compensación es una nueva acción de negocio que intenta neutralizar o reparar un efecto ya confirmado. También puede fallar, duplicarse o requerir intervención humana.



El historial conserva autorización, falla, anulación solicitada y resultado. Si el pago se capturó, se solicita reembolso en vez de anular la autorización.

Intuición: coreografía y orquestación

Coreografía

Como bailarines que conocen las señales: cada participante observa un hecho y decide su próximo movimiento, sin un director que conserve todo el plan.

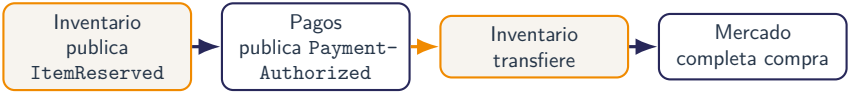
Orquestación

Como una dirección de escena: un coordinador conserva el estado del proceso y envía la próxima instrucción.

Son dos formas de coordinar la misma saga. Ninguna elimina estados durables, idempotencia, timeouts ni reparación.

Coreografía en Asteria: cada servicio reacciona a hechos

NÚCLEO



Ventaja

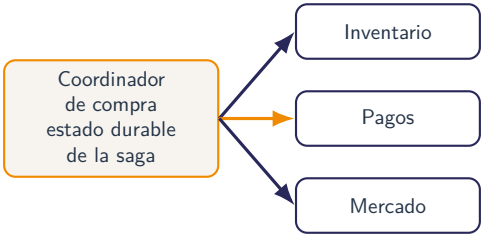
Participantes poco acoplados; nuevos lectores se agregan con facilidad.

Riesgo

El proceso global queda repartido entre contratos; ciclos y fallas son difíciles de visualizar.

Orquestación en Asteria: un coordinador conserva el proceso

NÚCLEO



Ventaja

Un lugar visible para estado, deadlines, reintentos, compensaciones e intervención humana.

Riesgo

Puede concentrar demasiada lógica; coordina, pero no debe apropiarse de los datos de otros.

Coreografía u orquestación: elegir por el proceso

TRADE-OFF

Dimensión	Coreografía	Orquestación
Próximo paso	cada servicio reacciona a hechos	coordinador envía comandos
Estado global	repartido	visible en el coordinador
Cambio de flujo	revisar varios contratos	modificar el proceso coordinador
Fallas largas/humanas	difíciles de seguir	fáciles de supervisar
Riesgo	ciclos y lógica emergente	coordinador con demasiada responsabilidad

Para la compra de #17, la orquestación suele ser más clara cuando hay pago incierto, compensaciones y revisión manual; Ranking y Notificaciones pueden continuar coreografiados.

UNIVERSIDAD AUSTRAL

INGENIERÍA

Departamento de Computación

Migrar y operar: la frontera debe sobrevivir en producción

Sistemas Distribuidos

¿Qué extraer primero en Asteria?

Ranking: candidato inicial

- carga diferente y con picos;
- tolera atraso acotado;
- puede construirse como proyección;
- falla sin detener la compra;
- escala y despliega de forma distinta.

Inventario: extracción riesgosa

- preserva propiedad única;
- participa en el camino crítico;
- genera sagas y estados intermedios;
- exige idempotencia y operación madura;
- una falla puede duplicar o bloquear objetos.

La primera frontera debe ofrecer beneficio y aprendizaje con riesgo controlable.

NÚCLEO

Migración gradual: patrón Strangler Fig

Intuición

En vez de reemplazar toda la aplicación en una fecha única, una parte nueva crece al costado de la anterior. El tráfico se desvía por porciones hasta retirar el camino viejo.

Definición

El patrón **Strangler Fig** es una migración incremental donde el componente antiguo y el nuevo coexisten y una fachada decide qué camino utilizar.

No significa simplemente "crear un microservicio". Es un método para reducir riesgo: coexistir, comparar, desviar y retirar.

Prueba gradual y vuelta atrás: limitar el riesgo

Prueba canary

Una fracción pequeña del tráfico real usa la versión nueva. Se comparan errores, latencia y resultados antes de ampliar.

Vuelta atrás de versión

Si las métricas empeoran, el router devuelve el tráfico al camino anterior. No deshace efectos de negocio ya confirmados.

La migración es reversible mientras ambos caminos coexistentes puedan recibir tráfico y los datos se mantengan comparables.

Migración gradual aplicada a Ranking

NÚCLEO



1. alimentar ambos cálculos;
2. comparar exactitud, atraso y costo;
3. enviar una fracción pequeña al camino nuevo;
4. volver al anterior si las métricas empeoran;
5. retirar el código viejo después de evidencia suficiente.

Balance de decisiones en Asteria

Decisión	Beneficio esperado	Qué cuesta sostener	Cómo comprobar
Réplicas y caché	lecturas más rápidas	recursos, invalidación, oferta potencialmente vieja	p99 y aciertos de caché
Ranking separado	compra menos expuesta al torneo	outbox, broker, atraso, duplicados, otra guardia	p99, lag e incidentes
Si Pagos se separa	controles y cambios de pago aislados	RPC, timeout ambiguo, IDs idempotentes	fallas de pago y auditoría
Si Inventario se separa	escala y versión propias	llamadas críticas, saga y recuperación de reservas	conflictos, latencia y compras correctas

CASO CONDUCTOR

Decisión: probar Ranking primero. Inventario queda unido mientras su costo distribuido supere el beneficio medido.

Vocabulario operativo I: CI, CD y preparación operativa

Término	Definición de trabajo
CI — Integración Continua	integrar cambios con frecuencia y ejecutar compilación, pruebas y controles automáticos
CD — Entrega Continua	mantener cada cambio validado listo para producción; la promoción puede ser manual
CD — Despliegue Continuo	poner automáticamente en producción todo cambio que supera los controles; hay que aclarar qué significado se usa
Preparación operativa (readiness)	evidencia de que un equipo puede publicar, observar, recuperar y sostener una capacidad en producción

CI y CD automatizan el camino del cambio; no sustituyen contratos compatibles, recuperación, métricas ni responsables de incidentes.

Vocabulario operativo II: guardia, procedimiento, SLO y trazas

Término	Definición de trabajo
Guardia (guardia)	rotación responsable de recibir alertas y coordinar la respuesta a incidentes
Procedimiento (runbook)	pasos escritos para diagnosticar, mitigar y recuperar una falla repetible
SLI	indicador medido, por ejemplo compras correctas o latencia p99
SLO	meta cuantificada para un SLI durante una ventana, por ejemplo 99,9% de compras correctas por mes
Trazas distribuidas	correlación del recorrido de una petición o evento a través de varios procesos

Estas capacidades hacen visible quién opera la frontera, qué promesa intenta cumplir y cómo investigar una falla entre servicios.

Preparación operativa: separar implica sostener otro sistema

Definición

Existe preparación operativa cuando el equipo demuestra que puede publicar, observar y recuperar la capacidad en producción; no alcanza con que un chequeo `/ready` responda.

- CI y publicación de versiones;
- vuelta atrás probada;
- permisos y secretos;
- logs, métricas y trazas;
- retries e idempotencia;
- SLO y alertas;
- guardia y procedimientos;
- backup y recuperación;
- pruebas de falla;
- capacidad y costos.

Sin esta preparación, separar puede demorar cambios y reducir confiabilidad aunque el diagrama parezca moderno.

NÚCLEO

Cómo leer las métricas de autonomía

Métrica	Qué significa y qué pregunta responde
Tiempo hasta producción (tiempo hasta producción)	desde que un cambio está listo hasta que funciona: ¿la frontera acelera el cambio?
Publicaciones sin coordinación	versiones que salen sin sincronizar otros equipos
Escala independiente	recursos agregados solo a la capacidad que crece
Alcance del incidente (alcance del incidente)	usuarios, regiones, datos y capacidades afectados
p99	valor por debajo del cual queda el 99% de observaciones; muestra la cola de latencia
Coordinaciones entre equipos	aprobaciones, tickets y bloqueos necesarios para cambiar u operar

Métricas para comprobar que la separación valió la pena

NÚCLEO

Métrica	Qué mide	Qué esperamos
Tiempo hasta producción	cambio listo a versión funcionando	bajar sin aumentar incidentes
Publicaciones sin coordinación	versiones que no esperan a otros equipos	aumentar
Escala por capacidad	recursos solo donde crece carga	evitar escalar todo Asteria
Alcance del incidente	usuarios y capacidades afectados	reducir
p99 y errores	experiencia de la cola y fallas	cumplir el SLO
Coordinaciones cruzadas	trabajo manual entre equipos	reducir

Hace falta una línea base anterior. Si el nuevo servicio sigue compartiendo base, versiones y guardias con la aplicación, la independencia prometida no se materializó.

Actividad: defender una frontera de Asteria

ACTIVIDAD

Elijan una capacidad

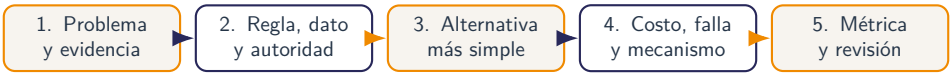
- inventario;
- pagos;
- ranking;
- chat;
- combate por zona.

Completen el argumento

1. regla e invariante;
2. responsable y dato autoritativo;
3. alternativa simple;
4. frontera propuesta;
5. RPC/evento;
6. falla y recuperación;
7. métrica de éxito.

Método reusable para decidir una frontera

NÚCLEO



Marco del curso

$D = (P, F, G, S, T)$: la tecnología aparece después de declarar problema, fallas, garantías y costo aceptado.

Cierre comparativo: por qué distribuir y cuándo no hacerlo

Capacidad	Límite del diseño unido	Separación propuesta	Prueba
Ranking	picos afectan la compra	proyección por eventos; versión propia	p99, atraso y tiempo hasta producción
Pagos	controles y proveedor frenan cambios	servicio propio si existe operación madura	errores, auditoría y coordinaciones
Inventario	la transacción local protege propiedad	mantener unido hasta justificar una saga	duplicaciones, latencia y carga

La aplicación unida deja de alcanzar cuando una necesidad medible persiste y no se resuelve con modularidad interna. Distribuir es aceptar un costo para resolverla; si las métricas no mejoran, la decisión debe revisarse.

Volvemos a la predicción inicial

ACTIVIDAD

Revise su primera respuesta y complete:

1. ¿La capacidad elegida tiene un lenguaje e invariante coherentes?
2. ¿Existe un responsable técnico y operacional?
3. ¿Qué autonomía concreta se obtendría?
4. ¿Qué falla nueva aparece al cruzar la red?
5. ¿Cómo se preserva la integridad del flujo?
6. ¿Qué métrica podría refutar su decisión?

Cambiar de opinión es evidencia de aprendizaje si ahora puede justificar mejor la frontera.

Volvemos al diagnóstico inicial

ACTIVIDAD

1. **Falso:** un monolito puede imponer módulos, APIs y responsabilidades claras.
2. **Falso:** un microservicio no es una clase remota; representa una capacidad con autonomía operativa.
3. **Falso:** servicios separados pueden seguir acoplados por base de datos, contratos y versiones coordinadas.
4. **Falso:** la base de datos y el broker son dos escrituras; una caída entre ambas deja inconsistencia.
5. **Falso:** compensar crea un nuevo efecto; las acciones previas siguen formando parte de la historia.

Síntesis: una frontera debe justificar su costo

NÚCLEO



Distribuir no es progreso por sí mismo. Una frontera vale cuando resuelve una necesidad concreta, la mejora aparece en métricas y el equipo puede operar las fallas nuevas.

Exit ticket

ACTIVIDAD

Responda brevemente:

1. ¿Qué diferencia una frontera semántica de una frontera de red?
2. ¿Qué autonomía concreta justificaría separar Ranking?
3. ¿Qué sabe Mercado después de un timeout de Inventario?
4. ¿Qué resuelve outbox y qué deja abierto?
5. ¿Por qué compensar no equivale a rollback?

Fuentes para profundizar

NÚCLEO

- Kleppmann, *Designing Data-Intensive Applications*: servicios, RPC, transacciones, streams y sistemas derivados.
- Evans, *Domain-Driven Design*: bounded contexts y lenguaje ubicuo.
- Fowler, *Monolith First*, *Microservice Trade-Offs* y *Strangler Fig*.
- Shopify Engineering: modular monolith, componentización y límites internos.
- AWS Prescriptive Guidance: transactional outbox y sagas.
- Richardson, *Microservices Patterns*: sagas, contratos y datos por servicio.
- DORA, *Accelerate State of DevOps*: tiempo hasta producción, frecuencia de puesta en producción y confiabilidad operacional.